

5.1. Working with JSON files

In this project, we'll be looking at tracking corporate bankruptcies in Poland. To do that, we'll need to get data that's been stored in a `JSON` file, explore it, and turn it into a `DataFrame` that we'll use to train our model.

```
In [1]: import gzip
import json
import pickle

import pandas as pd
from IPython.display import VimeoVideo
```

```
In [ ]: VimeoVideo("694158732", h="73c2fb4e4f", width=600)
```

Prepare Data

Open

The first thing we need to do is access the file that contains the data we need. We've done this using multiple strategies before, but this time around, we're going to use the command line.

```
In [4]: VimeoVideo("693794546", h="6e1fab0a5e", width=600)
```

Out[4]:

10:15



Task 5.1.1: Open a terminal window and navigate to the directory where the data for this project is located.

- [What's the Linux command line?](#)
- [Navigate a file system using the Linux command line.](#)

As we've seen in our other projects, datasets can be large or small, messy or clean, and complex or easy to understand. Regardless of how the data looks, though, it needs to be saved in a file somewhere, and when that file gets too big, we need to *compress* it. Compressed files are easier to store because they take up less space. If you've ever come across a `ZIP` file, you've worked with compressed data.

The file we're using for this project is compressed, so we'll need to use a file utility called `gzip` to open it up.

```
In [ ]: VimeoVideo("693794604", h="a8c0f15712", width=600)
```

Task 5.1.2: In the terminal window, locate the data file for this project and decompress it.

- [What's gzip?](#)
- [What's data compression?](#)
- [Decompress a file using gzip.](#)

```
In [ ]: VimeoVideo("693794641", h="d77bf46d41", width=600)
```

Explore

Now that we've decompressed the data, let's take a look and see what's there.

```
In [ ]: VimeoVideo("693794658", h="c8f1bba831", width=600)
```

Task 5.1.3: In the terminal window, examine the first 10 lines of `poland-bankruptcy-data-2009.json`.

- [Print lines from a file in the Linux command line.](#)

Does this look like any of the data structures we've seen in previous projects?

```
In [ ]: VimeoVideo("693794680", h="7f1302444b", width=600)
```

Task 5.1.4: Open `poland-bankruptcy-data-2009.json` by opening the `data` folder to the left and then double-clicking on the file. 🖱️

How is the data organized?

Curly brackets? Key-value pairs? It looks similar to a Python dictionary. It's important to note that JSON is not *exactly* the same as a dictionary, but a lot of the same concepts apply. Let's try reading the file into a DataFrame and see what happens.

```
In [5]: VimeoVideo("693794696", h="dd5b5ad116", width=600)
```

Out[5]:

01:55



Task 5.1.5: Load the data into a DataFrame.

- Read a JSON file into a DataFrame using pandas.

```
In [7]: # df = pd.read_json("data/poland-bankruptcy-data-2009.json")
# df.head()
```

```
In [8]: VimeoVideo("693794711", h="fdb009c4eb", width=600)
```

Out[8]:

07:10



Hmmm. It looks like something went wrong, and we're going to have to fix it. Luckily for us, there's an error message to help us figure out what's happening here:

ValueError: Mixing dicts with non-Series may lead to ambiguous ordering.

What should we do? That error sounds serious, but the world is big, and we can't possibly be the first people to encounter this problem. When you come across an error, copy the message into a search engine and see what comes back. You'll get lots of results. The web has lots of places to look for solutions to problems like this one, and [Stack Overflow](#) is one of the best. [Click here to check out a possible solution to our problem.](#)

There are three things to look for when you're browsing through solutions on Stack Overflow.

1. **Context:** A good question is specific; if you click through that link, you'll see that the person asks a **specific** question, gives some relevant information about their OS and hardware, and then offers the code that threw the error. That's important, because we need...
2. **Reproducible Code:** A good question also includes enough information for you to reproduce the problem yourself. After all, the only way to make sure the solution actually applies to your situation is to see if the code in the question throws the error you're having trouble with! In this case, the person included not only the code they used to get the error, but the actual error message itself. That would be useful on its own, but since you're looking for an actual solution to your problem, you're really looking for...
3. **An answer:** Not every question on Stack Overflow gets answered. Luckily for us, the one we've been looking at did. There's a big green check mark next to the first solution, which means that the person who asked the question thought that solution was the best one.

Let's try it and see if it works for us too!

```
In [9]: VimeoVideo("693794734", h="fecea6a81e", width=600)
```

```
Out[9]:
```

04:55

Task 5.1.6: Using a context manager, open the file `poland-bankruptcy-data-2009.json` and load it as a dictionary with the variable name `poland_data`.

- [What's a context manager?](#)
- [Open a file in Python.](#)
- [Load a JSON file into a dictionary using Python.](#)

```
In [11]: # Open file and load JSON
# write your code here
with open ("data/poland-bankruptcy-data-2009.json", "r") as read_file:
    poland_data = json.load(read_file)

print(type(poland_data))
```

```
<class 'dict'>
```

```
In [ ]: print(type(poland_data))
```

Okay! Now that we've successfully opened up our dataset, let's take a look and see what's there, starting with the keys. Remember, the **keys** in a dictionary are categories of things in a dataset.

```
In [12]: VimeoVideo("693794754", h="18e70f4225", width=600)
```

Out[12]:

01:00



Task 5.1.7: Print the keys for `poland_data`.

- List the keys of a dictionary in Python.

```
In [13]: # Print `poland_data` keys
poland_data.keys()
```

```
Out[13]: dict_keys(['schema', 'data', 'metadata'])
```

`schema` tells us how the data is structured, `metadata` tells us where the data comes from, and `data` is the data itself.

Now let's take a look at the values. Remember, the **values** in a dictionary are ways to describe the variable that belongs to a key.

```
In [14]: VimeoVideo("693794768", h="8e5b53b0ca", width=600)
```

Out[14]:

03:00



Task 5.1.8: Explore the values associated with the keys in `poland_data`. What do each of them represent? How is the information associated with the `"data"` key organized?

```
In [15]: # Continue Exploring `poland_data`  
# poland_data["metadata"]  
# poland_data["schema"].keys()  
poland_data["data"][0]
```

```
Out[15]: {'company_id': 1,
          'feat_1': 0.17419,
          'feat_2': 0.41299,
          'feat_3': 0.14371,
          'feat_4': 1.348,
          'feat_5': -28.982,
          'feat_6': 0.60383,
          'feat_7': 0.21946,
          'feat_8': 1.1225,
          'feat_9': 1.1961,
          'feat_10': 0.46359,
          'feat_11': 0.21946,
          'feat_12': 0.53139,
          'feat_13': 0.14233,
          'feat_14': 0.21946,
          'feat_15': 592.24,
          'feat_16': 0.6163,
          'feat_17': 2.4213,
          'feat_18': 0.21946,
          'feat_19': 0.12272,
          'feat_20': 37.573,
          'feat_21': 0.9969,
          'feat_22': 0.2951,
          'feat_23': 0.097402,
          'feat_24': 0.75641,
          'feat_25': 0.46359,
          'feat_26': 0.50669,
          'feat_27': 1.9737,
          'feat_28': 0.32417,
          'feat_29': 5.9473,
          'feat_30': 0.22493,
          'feat_31': 0.12272,
          'feat_32': 100.82,
          'feat_33': 3.6203,
          'feat_34': 0.71453,
          'feat_35': 0.2951,
          'feat_36': 1.8079,
          'feat_37': 123140.0,
          'feat_38': 0.46359,
          'feat_39': 0.16501,
          'feat_40': 0.21282,
          'feat_41': 0.041124,
          'feat_42': 0.16501,
          'feat_43': 95.682,
          'feat_44': 58.109,
          'feat_45': 0.94621,
          'feat_46': 0.90221,
          'feat_47': 44.941,
          'feat_48': 0.26003,
          'feat_49': 0.1454,
          'feat_50': 1.348,
          'feat_51': 0.41299,
          'feat_52': 0.27622,
          'feat_53': 1.0457,
          'feat_54': 1.0458,
          'feat_55': 127280.0,
```

```
'feat_56': 0.16396,  
'feat_57': 0.37574,  
'feat_58': 0.83604,  
'feat_59': 6.5269e-06,  
'feat_60': 9.7145,  
'feat_61': 6.2813,  
'feat_62': 84.291,  
'feat_63': 4.3303,  
'feat_64': 4.0341,  
'bankrupt': False}
```

This dataset includes all the information we need to figure whether or not a Polish company went bankrupt in 2009. There's a bunch of features included in the dataset, each of which corresponds to some element of a company's balance sheet. You can explore the features by looking at the [data dictionary](#). Most importantly, we also know whether or not the company went bankrupt. That's the last key-value pair.

Now that we know what data we have for each company, let's take a look at how many companies there are.

```
In [16]: VimeoVideo("693794783", h="8d333027cc", width=600)
```

```
Out[16]:
```

01:10



Task 5.1.9: Calculate the number of companies included in the dataset.

- Calculate the length of a list in Python.
- List the keys of a dictionary in Python.

```
In [17]: # Calculate number of companies  
len(poland_data["data"])
```

```
Out[17]: 9977
```

And then let's see how many features were included for one of the companies.

```
In [18]: VimeoVideo("693794797", h="3c1eff82dc", width=600)
```

Out[18]:

01:41



Task 5.1.10: Calculate the number of features associated with "company_1" .

```
In [19]: # Calculate number of features
len(poland_data["data"][0])
```

Out[19]: 66

Since we're dealing with data stored in a JSON file, which is common for semi-structured data, we can't assume that all companies have the same features. So let's check!

```
In [20]: VimeoVideo("693794810", h="80e195944b", width=600)
```

Out[20]:

02:56



Task 5.1.11: Iterate through the companies in `poland_data["data"]` and check that they all have the same number of features.

- What's an iterator?
- Access the items in a dictionary in Python.
- Write a for loop in Python.

```
In [ ]: # Iterate through companies
for item in poland_data["data"]:
    if len(item) != 66:
        print("ALERT")
```

It looks like they do!

Let's put all this together. First, open up the compressed dataset and load it directly into a dictionary.

```
In [21]: VimeoVideo("693794824", h="dbfc9b43ee", width=600)
```

Out[21]:

03:04



Task 5.1.12: Using a context manager, open the file `poland-bankruptcy-data-2009.json.gz` and load it as a dictionary with the variable name `poland_data_gz`.

- What's a context manager?
- Open a file in Python.
- Load a JSON file into a dictionary using Python.

```
In [24]: # Open compressed file and load contents
# write your code here
with gzip.open("data/poland-bankruptcy-data-2009.json.gz", "r") as read_file:
    poland_data_gz = json.load(read_file)
```

```
In [25]: print(type(poland_data_gz))
```

```
<class 'dict'>
```

Since we now have two versions of the dataset — one compressed and one uncompressed — we need to compare them to make sure they're the same.

```
In [26]: VimeoVideo("693794837", h="925b5e4e5a", width=600)
```

```
Out[26]:
```

01:52



Task 5.1.13: Explore `poland_data_gz` to confirm that it contains the same data as `data`, in the same format.

```
In [27]: # Explore `poland_data_gz`
print(poland_data_gz.keys())
print(len(poland_data_gz["data"]))
print(len(poland_data_gz["data"][0]))
```

```
dict_keys(['schema', 'data', 'metadata'])
9977
66
```

Looks good! Now that we have an uncompressed dataset, we can turn it into a DataFrame using `pandas`.

```
In [28]: VimeoVideo("693794853", h="b74ef86783", width=600)
```

Out[28]:

03:28

Task 5.1.14: Create a DataFrame `df` that contains the all companies in the dataset, indexed by `"company_id"`. Remember the principles of *tidy data* that you learned in Project 1, and make sure your DataFrame has shape `(9977, 65)`.

- Create a DataFrame from a dictionary in pandas.

```
In [30]: # write you code here
df = pd.DataFrame().from_dict(poland_data_gz["data"]).set_index("company_id")

print(df.shape)
df.head()
```

(9977, 65)

```
Out[30]:
```

	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8	feat_9
company_id									
1	0.174190	0.41299	0.14371	1.3480	-28.9820	0.60383	0.219460	1.12250	1.196
2	0.146240	0.46038	0.28230	1.6294	2.5952	0.00000	0.171850	1.17210	1.601
3	0.000595	0.22612	0.48839	3.1599	84.8740	0.19114	0.004572	2.98810	1.007
5	0.188290	0.41504	0.34231	1.9279	-58.2740	0.00000	0.233580	1.40940	1.339
6	0.182060	0.55615	0.32191	1.6045	16.3140	0.00000	0.182060	0.79808	1.812

5 rows × 65 columns

Import

Now that we have everything set up the way we need it to be, let's combine all these steps into a single function that will decompress the file, load it into a DataFrame, and return it to us as something we can use.

```
In [ ]: VimeoVideo("693794879", h="f51a3a342f", width=600)
```

Task 5.1.15: Create a `wrangle` function that takes the name of a compressed file as input and returns a tidy DataFrame. After you confirm that your function is working as intended, submit it to the grader.

```
In [31]: def wrangle(filename):  
         return df
```

```
In [32]: df = wrangle("data/poland-bankruptcy-data-2009.json.gz")  
         print(df.shape)  
         df.head()
```

(9977, 65)

```
Out[32]:
```

	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8	feat_9
company_id									
1	0.174190	0.41299	0.14371	1.3480	-28.9820	0.60383	0.219460	1.12250	1.196
2	0.146240	0.46038	0.28230	1.6294	2.5952	0.00000	0.171850	1.17210	1.601
3	0.000595	0.22612	0.48839	3.1599	84.8740	0.19114	0.004572	2.98810	1.007
5	0.188290	0.41504	0.34231	1.9279	-58.2740	0.00000	0.233580	1.40940	1.339
6	0.182060	0.55615	0.32191	1.6045	16.3140	0.00000	0.182060	0.79808	1.812

5 rows × 65 columns



Copyright 2023 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.